

PRESENTACIÓN

Presentado por:

Patricio Olguin

Oscar Leyton

Benjamín Vásquez

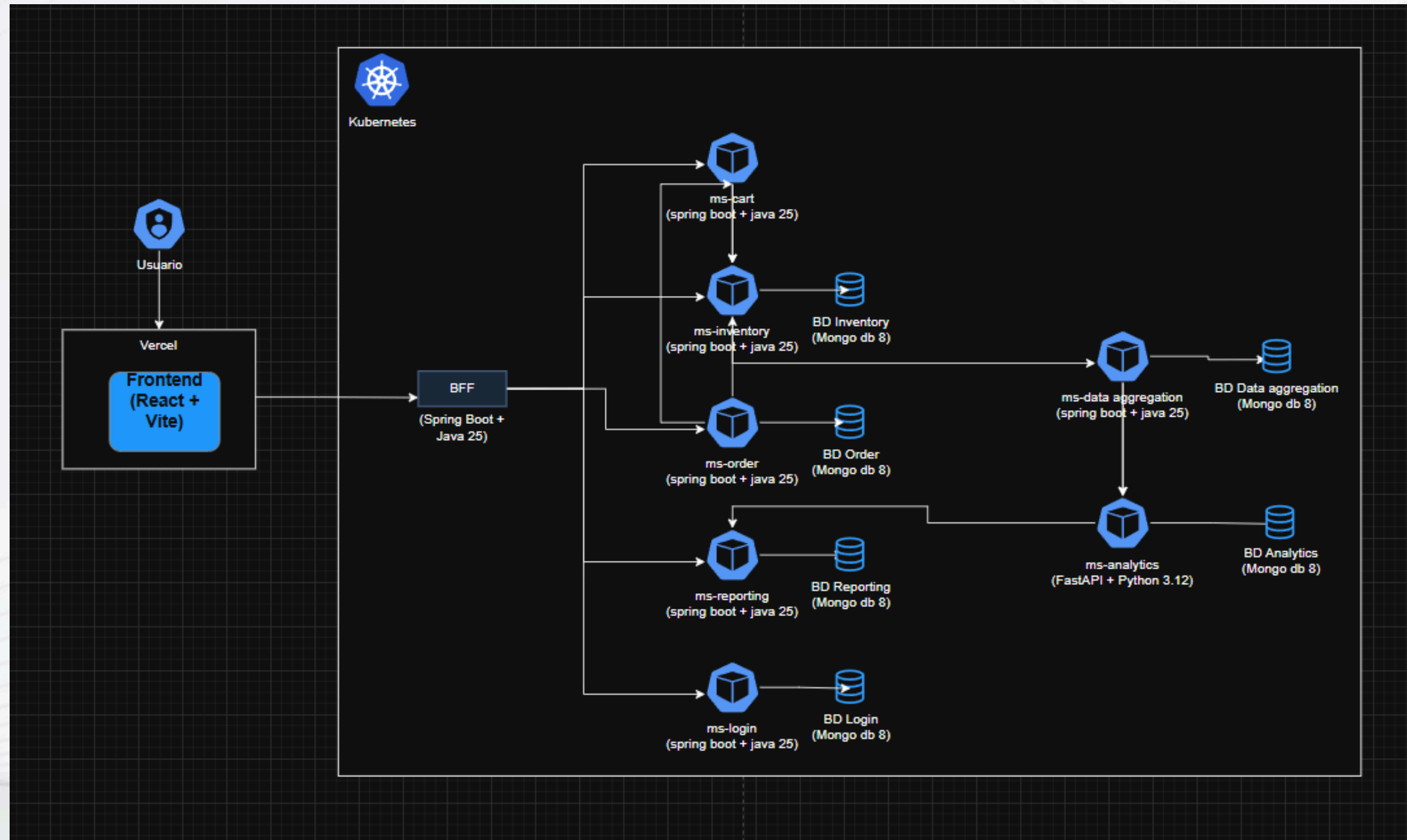
INTRODUCCIÓN

Grupo Cordillera, empresa dedicada al retail y comercialización de productos para el hogar y tecnología, enfrenta desafíos asociados a la dispersión de datos en múltiples plataformas, dificultando la obtención de información consolidada para la toma de decisiones estratégicas.

Con el propósito de resolver esta problemática, se propone el desarrollo de una plataforma de monitoreo inteligente basada en una arquitectura de microservicios, capaz de integrar información proveniente de distintos sistemas organizacionales.

La solución contempla la gestión centralizada de datos, el análisis de indicadores clave de desempeño (KPI) y la visualización de reportes en tiempo real, proporcionando a la alta dirección una herramienta moderna, escalable y confiable para apoyar la gestión empresarial.

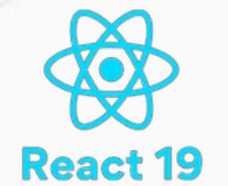
DIAGRAMA GENERAL



STACK TECNOLÓGICO

Backend

- Java 25
- Spring Boot 4
- FastAPI
- JWT
- OpenFeign
- Python 3.12



Frontend

- React 19
- TypeScript
- Vite
- Axios



Infraestructura y Datos

- MongoDB
- Docker
- Kubernetes
- Swagger/OpenAPI
- GlitchTip



ARQUITECTURA Y PATRONES DE DISEÑO

Arquetipos

Backend:

- Controller
- Service
- Repository

Frontend:

- Components
- Services
- Contexts

Patrones arquitectonicos

- Arquitectura en capas
- Frontend React
- BFF
- Microservicios
- Databaser per service
- Kubernetes + Docker

Patrones de diseño

Backend:

- Repository Pattern
- Factory method

Frontend:

- Provider Pattern

SEGURIDAD Y AUTENTICACIÓN

La seguridad de la plataforma se implementó mediante JWT y Spring Security. Cuando un usuario inicia sesión, el microservicio ms-login valida sus credenciales y genera un token JWT que contiene información del usuario y su rol.

El frontend almacena este token utilizando AuthProvider y localStorage, permitiendo mantener la sesión activa incluso después de actualizar la página. Posteriormente, cada solicitud incluye automáticamente el encabezado Authorization: Bearer Token, el cual es propagado por el BFF hacia los microservicios internos.

Este mecanismo permite proteger los endpoints sensibles y aplicar control de acceso basado en roles (ADMIN, VENTAS y CLIENTE), garantizando que cada usuario acceda únicamente a los recursos autorizados.

Spring Security

JWT (JSON Web
Token)

Axios Interceptors

AuthContext y
AuthProvider (React)

BFF (Backend For
Frontend)

KUBERNETES Y DESPLIEGUE

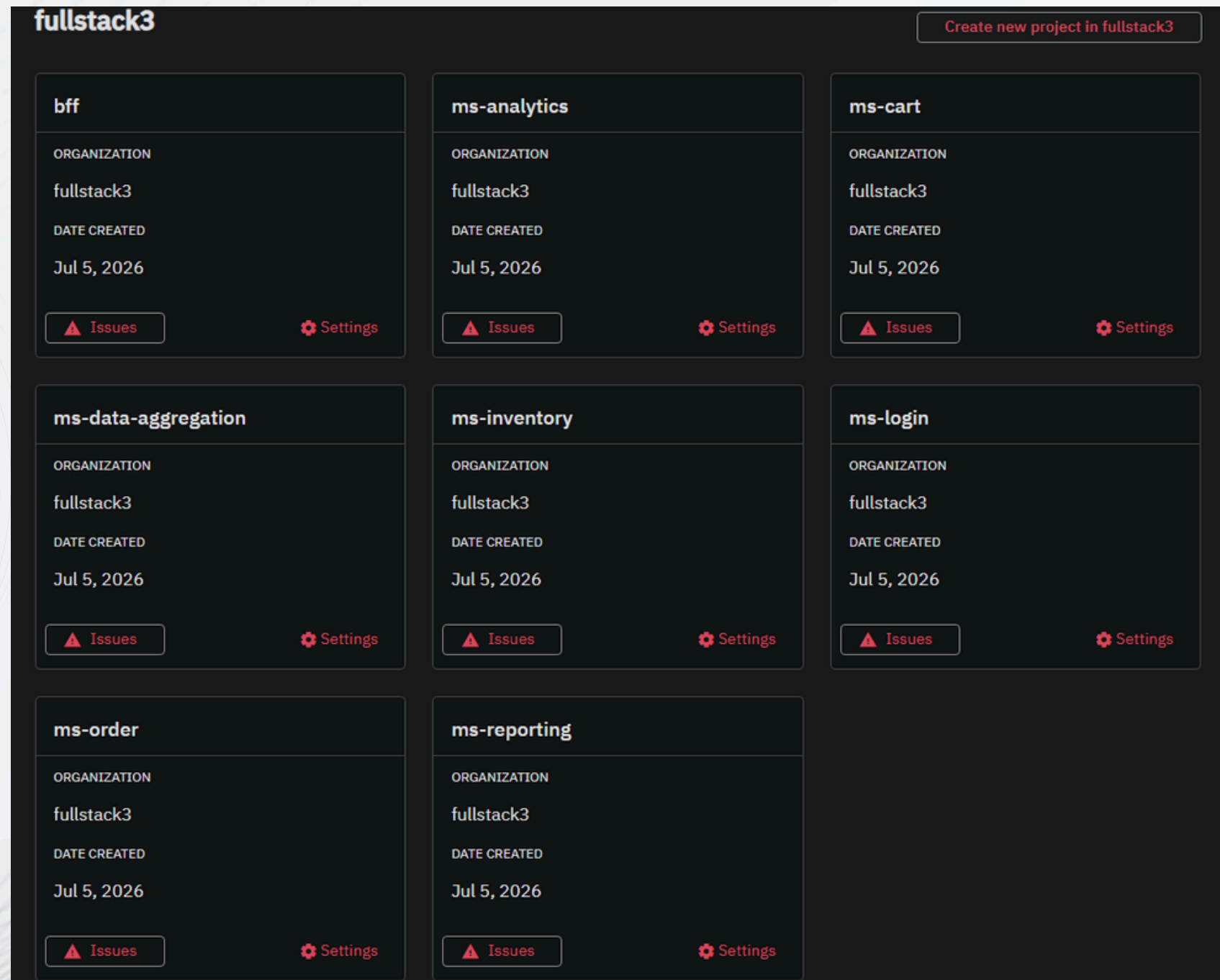
La aplicación fue contenerizada utilizando Docker, permitiendo empaquetar cada microservicio junto con sus dependencias para asegurar un comportamiento consistente en cualquier entorno.

Para la orquestación se utilizó Kubernetes, donde cada microservicio se ejecuta en un Pod independiente y se expone mediante Services

Esta arquitectura permite mejorar la escalabilidad, la alta disponibilidad, la tolerancia a fallos y la facilidad de despliegue, ya que los servicios pueden actualizarse o escalarse de manera independiente sin afectar el funcionamiento general de la plataforma.

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	k8s_mongodb_mongodb-78497dd8-q9vqj_defi	ec7483cad7c2	49f1d7b87c2d		0.45%	39 minutes ago	
☐	k8s_ms-login_ms-login-6c4fd674bf-nft6f_defa	3359862260a6	ee81aca21c7e		0.03%	39 minutes ago	
☐	k8s_ms-reporting_ms-reporting-6789d49b5-tvl	df0062ed0b62	2f64ec9ce323		0.04%	39 minutes ago	
☐	k8s_ms-order_ms-order-8cccd7fdc-q6qsc_defi	021878e37258	fd423d644400		0.09%	39 minutes ago	
☐	k8s_ms-analytics_ms-analytics-69cb96744f-fp	968dea68628f	b1bcbdfd524e		0.16%	39 minutes ago	
☐	k8s_frontend_frontend-7cd7f85d75-975qn_defi	fb67c780e838	c05a95865e16		0%	39 minutes ago	
☐	k8s_ms-inventory_ms-inventory-84ddd767f7-h	39f33cbe15c2	0c78db32060d		0.04%	39 minutes ago	
☐	k8s_bff_bff-675c5fd946-tnkf6_default_28afc8l	28ca8970031d	06bd02e28d05		0.05%	39 minutes ago	
☐	k8s_ms-data-aggregation_ms-data-aggregatio	ab990ed33658	136dde7aeee3		0.04%	39 minutes ago	
☐	k8s_ms-cart_ms-cart-67cffdfdb8-tbpws_defau	97984d94bc55	de83080149fa		0.04%	39 minutes ago	

GLITCHTIP



Para el monitoreo de errores en produccion se integro GlitchTip donde cada uno de los microservicios y el BFF estan configurados como proyectos independientes dentro de GlitchTip, lo que permite capturar y centralizar las excepciones de cada componente de forma aislada

Este proyecto nos permitió aplicar de manera práctica los conocimientos de desarrollo Full Stack y arquitectura de microservicios, utilizando tecnologías modernas.

La solución fue diseñada siguiendo principios de escalabilidad, mantenibilidad y seguridad.

Para centralizar el acceso desde el frontend y servicios especializados para análisis y generación de reportes.

CONCLUSIÓN

Además, incorporamos herramientas de automatización y despliegue como Docker y Kubernetes, junto con migraciones de base de datos mediante Mongock, lo que nos permitió construir una plataforma preparada para crecer y adaptarse a futuras necesidades del negocio.

En definitiva, el proyecto no solo cumple con los requerimientos funcionales planteados, sino que también demuestra patrones de diseño y arquitecturas modernas utilizadas actualmente en la industria.

The background features a series of overlapping, wavy, horizontal bands in shades of light blue, green, and yellow. These bands are set against a white background. In the top-left and bottom-right corners, there are clusters of small, light-colored dots. The overall design is modern and minimalist.

**MUCHAS
GRACIAS**